Chapter 10: Services

1. Overview

A **service** in Angular is not a special construct enforced by the compiler — it is a plain TypeScript class that carries out a well-defined piece of work: fetching data, holding shared state, wrapping browser APIs, logging, caching, orchestrating business logic. What makes it "Angular-aware" is almost always the `@Injectable()` decorator plus Angular's **Dependency Injection (DI)** system, which is responsible for constructing the service, resolving its own dependencies, and handing the same (or a new) instance to whoever asks for it.

Services exist to solve one structural problem: **components should describe the view, not own the logic**. If a component fetches data, transforms it, caches it, and also renders a template, it becomes untestable and unreusable. Pulling that logic into a service gives you:

- **Separation of concerns** — components stay declarative and thin.
- **Reusability** — the same logic is consumed by many components without duplication.
- **Cross-component communication** — a shared, injected instance becomes a single source of truth that unrelated components can read and write to, without prop-drilling through `@Input` / `@Output` chains.
- **Testability** — a service is a plain class; you can `new` it up or mock it without touching the DOM or the component-testing harness.

This chapter covers what a service actually is, how `@Injectable` and `providedIn` control singleton behavior and lazy instantiation, how to design services for shared state, when to scope a service narrowly instead of to `root`, and how to unit test them properly — including the subscription-leak pitfalls that show up constantly in interviews and in production bugs.

---

2. Core Concepts

2.1 What a Service Actually Is

A service is just a class:

```
export class LoggerService {
  log(message: string): void {
    console.log(`[LOG]: ${message}`);
  }
}
```

Nothing here is Angular-specific. You could `new LoggerService().log('hi')` in plain Node. Angular's contribution is *how instances of this class get created and shared* — that is entirely the job of the injector, not the class itself.

2.2 `@Injectable()` — What It Really Does

```
import { Injectable } from '@angular/core';

@Injectable({
  providedIn: 'root'
})
export class LoggerService {
  constructor(private http: HttpClient) {}
}
```

`@Injectable()` does two distinct things:

1. **It marks the class as a DI target**, attaching design-time metadata (via TypeScript's `emitDecoratorMetadata` + `reflect-metadata`, or Ivy's compile-time analysis) so Angular knows *what to inject into the class's own constructor*. Without `@Injectable()`, if the class's constructor asks for other injectable dependencies, Angular cannot reliably determine their types when it tries to construct the class — you'd get a runtime DI error, especially in AOT and stricter Ivy builds.
2. `providedIn` **registers the class with a specific injector** without requiring you to list it in any NgModule's `providers` array. This is called "tree-shakable providers" — if nothing ever injects `LoggerService`, Ivy's tree-shaker can drop it from the bundle entirely, because the provider registration lives *on the class itself*, not in a module's static provider list.

A service *can* technically omit `@Injectable()` if it has zero constructor dependencies (Angular can call `new Foo()` fine), but this is fragile — add a dependency later and it silently breaks under certain compilation configurations. **Always decorate every injectable class**, even with no dependencies, as a matter of discipline.

2.3 `providedIn` Values

Value	Meaning
`'root'`	Singleton for the whole application, registered on the root `EnvironmentInjector`. Tree-shakable. This is the default recommendation for 95% of services.
`'platform'`	Shared across multiple Angular applications on the same page (rare — micro-frontend scenarios).
`'any'`	A new instance per lazy-loaded module that injects it; eagerly-loaded parts of the app still share one root instance. Rare, mostly legacy.
A specific `NgModule` class	Scopes the service to that module's injector — legacy pattern, mainly seen pre-standalone-APIs or when deliberately isolating state to a feature module.
Omitted, service listed in a component's `providers: []`	Scoped to that component (and its children) — a **new instance per component instance**.

2.4 The Singleton Pattern in Angular

"Singleton" in Angular does **not** mean "exactly one instance ever exists in the process," the way a classic GoF singleton does. It means **one instance per injector**. Angular has a *hierarchy* of injectors:

```
Root EnvironmentInjector (providedIn: 'root' lives here)
  └── Platform / Lazy-loaded feature EnvironmentInjector (for lazy routes)
        └── ElementInjector hierarchy (per component, from providers: [])
```

When a component (or anything) asks for a service via constructor injection, Angular walks **up** the injector tree starting from the requesting component's own element injector, looking for a matching provider. The first injector in that upward walk that has a registration wins, and *that* injector caches the instance so the next lookup from the same injector (or its children) returns the same object.

This means:

- `providedIn: 'root'` genuinely gives you one instance for the whole app (in the common case with no lazy-loaded module overrides).
- A service listed in a **component's** `providers` array gives that component (and everything nested inside it) its own fresh instance, separate from the root singleton — even if the class itself used `providedIn: 'root'` as its default, a component-level `providers` entry **overrides** and shadows it for that subtree.

2.5 Service Design for Shared State / Cross-Component Communication

The most interview-relevant use of services is as a **shared state container** between components that have no direct parent-child relationship (siblings, or components deep in unrelated branches of the tree).

The canonical pattern:

1. Store state privately in the service (often as a `BehaviorSubject` or Angular Signal).
2. Expose a **read-only** public surface (`Observable` via `.asObservable()`, or a read-only `Signal`).
3. Expose explicit **mutation methods** (`addItem()`, `setUser()`, `clear()`) rather than a public `.next()` — this keeps a single, auditable path for state changes and stops components from pushing arbitrary values into your stream.

This is effectively a lightweight, hand-rolled Redux/NgRx-store pattern, and interviewers frequently ask you to design exactly this from scratch (see the Code Examples section).

2.6 Service Composition

Services can inject other services — this is normal and encouraged. It lets you build small, single-purpose services and compose them into higher-level ones (a `CartService` might depend on `ProductService` and `AuthService`). Angular's DI resolves this dependency graph automatically and lazily, and because everything funnels through `providedIn: 'root'`, there's no manual wiring needed — no service locator, no manual factory calls.

Composition pitfalls:

- **Circular dependencies** between two `root`-provided services (A injects B, B injects A) throw a runtime

error (`NG0200: Circular dependency in DI detected`) — refactor shared logic into a third service both depend on.

- Prefer composing **small, focused services** over one "God service" holding unrelated concerns (auth + logging + cart) — it becomes an untestable dumping ground and creates unnecessary coupling for anything that only needs one slice of it.

2.7 Component-Scoped vs Root-Scoped — Decision Criteria

Scope to `root` when…	**Scope to a component when…**
State must be shared/visible app-wide (auth session, current user, feature flags).	State is local to one UI feature and must **not** leak between sibling instances (e.g., a wizard's draft state, a modal's local form state).
The service is stateless (pure utility/formatting/http wrapper) — no reason to duplicate it.	You render the **same component multiple times** on one page (e.g., a `<product-card>` repeated in an `*ngFor`) and each instance needs its **own** isolated instance of an injected service (e.g., a per-card `ExpandStateService`).
You want it to persist across route navigations within the app's lifetime.	You want the instance to be **destroyed and garbage-collected** automatically when the component is destroyed (its element injector — and everything scoped to it — is torn down with the component, cleaning up subscriptions if `ngOnDestroy` is implemented).
You need exactly one instance to avoid inconsistent state across the app.	You are deliberately isolating state per feature instance to avoid cross-instance bleed (classic bug: a shared "selected tab" service unintentionally shared between two unrelated tab-group components on the same page).

The interview-ready one-liner: **root-scope for true app-wide singletons and stateless utilities; component-scope when you need one instance per UI instance, with automatic cleanup tied to that component's lifecycle.**

2.8 Testing Services

Because a service is a plain class, it can be tested two ways:

1. **Plain instantiation** (no Angular test bed needed) if it has simple/mockable constructor dependencies:

   ```
   const service = new LoggerService();
   ```

2. **`TestBed`-based DI resolution** — preferred when the service itself has injected dependencies (especially `HttpClient`, `Router`, other services) because `TestBed` lets you provide mocks/spies through the same DI mechanism the real app uses, and lets you use `HttpClientTestingModule` / `provideHttpClientTesting()` to intercept HTTP calls.

`TestBed.inject(MyService)` mirrors production DI resolution exactly, which is why it's the standard approach for anything beyond a trivial, dependency-free class.

---

3. Code Examples

3.1 Basic `@Injectable` Service with `providedIn: 'root'`

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';

export interface Product {
  id: number;
  name: string;
  price: number;
}

@Injectable({
  providedIn: 'root'
})
export class ProductService {
  private readonly apiUrl = '/api/products';

  constructor(private http: HttpClient) {}

  getAll(): Observable<Product[]> {
    return this.http.get<Product[]>(this.apiUrl);
  }

  getById(id: number): Observable<Product> {
    return this.http.get<Product>(`${this.apiUrl}/${id}`);
  }
}
```

3.2 Realistic Shared-State Service (Cross-Component Communication)

This is the pattern most often asked for in interviews: a cart shared between an unrelated `ProductListComponent` and `CartBadgeComponent`, neither of which is the other's parent.

```
import { Injectable } from '@angular/core';
import { BehaviorSubject, Observable } from 'rxjs';

export interface CartItem {
  productId: number;
  name: string;
  quantity: number;
  price: number;
}

@Injectable({
  providedIn: 'root'
})
export class CartService {
```

```
  // Private, mutable source of truth — never exposed directly.
  private readonly itemsSubject = new BehaviorSubject<CartItem[]>([]);

  // Public, read-only stream — consumers can subscribe but cannot push values.
  readonly items$: Observable<CartItem[]> = this.itemsSubject.asObservable();

  // Derived state, computed on demand rather than duplicated/stored separately.
  get itemCount(): number {
    return this.itemsSubject.value.reduce((sum, i) => sum + i.quantity, 0);
  }

  addItem(item: CartItem): void {
    const current = this.itemsSubject.value;
    const existing = current.find(i => i.productId === item.productId);

    const updated = existing
      ? current.map(i =>
          i.productId === item.productId
            ? { ...i, quantity: i.quantity + item.quantity }
            : i
        )
      : [...current, item];

    this.itemsSubject.next(updated);
  }

  removeItem(productId: number): void {
    this.itemsSubject.next(
      this.itemsSubject.value.filter(i => i.productId !== productId)
    );
  }

  clear(): void {
    this.itemsSubject.next([]);
  }
}
```

Consumer A — adds items:

```
@Component({
  selector: 'app-product-list',
  template: `
    <button *ngFor="let p of products" (click)="add(p)">Add {{ p.name }}</button>
  `
})
export class ProductListComponent {
  products: Product[] = [
    { id: 1, name: 'Widget', price: 9.99 },
    { id: 2, name: 'Gadget', price: 19.99 }
  ];

  constructor(private cart: CartService) {}
```

```
  add(product: Product): void {
    this.cart.addItem({ productId: product.id, name: product.name, quantity: 1, price:
product.price });
  }
}
```

Consumer B — completely unrelated component, reads the same singleton:

```
@Component({
  selector: 'app-cart-badge',
  template: `<span class="badge">{{ count$ | async }}</span>`
})
export class CartBadgeComponent {
  count$ = this.cart.items$.pipe(
    map(items => items.reduce((sum, i) => sum + i.quantity, 0))
  );

  constructor(private cart: CartService) {}
}
```

Because `CartService` is `providedIn: 'root'`, both components resolve to the **same instance** — `CartBadgeComponent` reacts instantly to changes made in `ProductListComponent` with zero `@Input` / `@Output` wiring and no shared parent.

3.3 Modern Signal-Based Shared State (Angular 16+)

```
import { Injectable, signal, computed } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class CartSignalService {
  private readonly itemsSignal = signal<CartItem[]>([]);

  readonly items = this.itemsSignal.asReadonly();
  readonly itemCount = computed(() =>
    this.itemsSignal().reduce((sum, i) => sum + i.quantity, 0)
  );

  addItem(item: CartItem): void {
    this.itemsSignal.update(current => {
      const existing = current.find(i => i.productId === item.productId);
      return existing
        ? current.map(i => i.productId === item.productId ? { ...i, quantity: i.quantity + 1
} : i)
        : [...current, item];
    });
  }
}
```

3.4 Component-Scoped Service (Deliberate Non-Singleton)

```
@Injectable() // note: no providedIn — this class is NOT auto-registered anywhere
export class TabStateService {
  private activeIndexSubject = new BehaviorSubject(0);
  activeIndex$ = this.activeIndexSubject.asObservable();

  select(index: number): void {
    this.activeIndexSubject.next(index);
  }
}

@Component({
  selector: 'app-tab-group',
  providers: [TabStateService], // <-- new instance per <app-tab-group> on the page
  template: `...`
})
export class TabGroupComponent {
  constructor(public tabState: TabStateService) {}
}
```

Two `<app-tab-group>` elements on the same page each get their **own** `TabStateService` instance — selecting a tab in one does not affect the other. If this service had instead been `providedIn: 'root'`, both tab groups would incorrectly share one active tab index.

3.5 Service Composition Example

```
@Injectable({ providedIn: 'root' })
export class AuthService {
  private userSubject = new BehaviorSubject<User | null>(null);
  user$ = this.userSubject.asObservable();

  setUser(user: User): void {
    this.userSubject.next(user);
  }
}

@Injectable({ providedIn: 'root' })
export class CartApiService {
  // Composes AuthService to attach the current user to every cart request.
  constructor(private auth: AuthService, private http: HttpClient) {}

  syncCart(items: CartItem[]): Observable<void> {
    return this.auth.user$.pipe(
      take(1),
      switchMap(user => this.http.post<void>(`/api/users/${user?.id}/cart`, items))
    );
  }
}
```

4. Internal Working

4.1 `providedIn` and the DI System

When you write:

```
@Injectable({ providedIn: 'root' })
export class MyService { }
```

The Angular compiler (Ivy) generates a static property on the class itself:

```
MyService.ɵprov = ɵɵdefineInjectable({
  token: MyService,
  factory: () => new MyService(...deps),
  providedIn: 'root'
});
```

This `ɵprov` (injectable definition) is attached directly to the class — it is **not** registered in any module's provider array at all. This is the mechanical basis of tree-shakable providers: the registration travels with the class, so if the bundler's tree-shaker determines the class is never imported/referenced anywhere reachable from the app's entry point, both the class *and* its provider registration are eliminated from the final bundle. Compare this to the pre-Ivy/legacy pattern of listing every service in `@NgModule({ providers: [MyService] })` — that approach could not be tree-shaken because the module's static provider list referenced the class regardless of whether anything actually injected it.

4.2 How the Singleton Instance Actually Gets Created — Lazy Instantiation

Angular does **not** instantiate `root`-provided services at bootstrap. Instantiation is strictly demand-driven:

1. At application start, the root `EnvironmentInjector` is created, but it holds only **provider definitions** (recipes: "if asked for token `MyService`, run this factory function") — no actual instances yet.
2. The first time *anything* (a component constructor, another service's constructor, an `inject()` call) asks the injector for `MyService`, the injector:
    - Checks its internal record/cache for an already-created instance for that token. None exists yet.
    - Looks up the provider definition (`ɵprov`) for that token.
    - Recursively resolves the constructor parameters of `MyService` first (each of *those* tokens goes through this exact same lazy-resolution process — DI resolution is fundamentally a recursive, depth-first walk of the dependency graph).
    - Invokes the factory function (`new MyService(...resolvedDeps)`).
    - **Caches** the resulting instance against that token in the injector's record table.
3. Every subsequent request for `MyService` **from that same injector or from a descendant injector that does not have its own competing provider**, returns the cached instance directly — no re-instantiation, no re-running of the constructor.

This is why: if a service is *never injected anywhere* in a running session (e.g., an admin-only service that no component in a regular user's session ever references), its constructor genuinely never runs and no instance is ever allocated — laziness is a real memory/perf behavior, not just a theoretical nuance.

4.3 The Injector Hierarchy Walk

For any given injection request, Angular resolves it by walking **upward**:

```
Requesting component's ElementInjector
   → parent component's ElementInjector (if providers[] present there)
      → ... up through ancestor ElementInjectors ...
         → the EnvironmentInjector for the current lazy-loaded module (if any)
            → the root EnvironmentInjector
```

The walk stops at the **first injector that has a provider registered for that exact token**. This explains the shadowing behavior in §2.4/§3.4: a component-level `providers: [TabStateService]` entry creates a *new* provider record on that component's own `ElementInjector`, which is found before the walk ever reaches the root injector's `ɵprov`-based registration — so the component subtree gets its own instance, and the root registration (if the class also had `providedIn: 'root'`) is simply never reached for anything under that component.

4.4 Instance Destruction

An `EnvironmentInjector` or `ElementInjector` is destroyed when its owning scope is destroyed — a component's `ElementInjector` is torn down when the component is destroyed (removed from the DOM by structural directives, route navigation away from a component that had it in `providers`, etc.), and a lazy-loaded module's injector can be destroyed if the associated route is configured to not be retained. When an injector is destroyed, Angular calls `ngOnDestroy()` on any of its cached provider instances that implement `OnDestroy` — this is your hook to unsubscribe, clear timers, and close connections (see §5.2). The root `EnvironmentInjector`, by contrast, lives for the entire application lifetime and is destroyed only when the whole Angular application is destroyed (e.g., `ApplicationRef.destroy()`), which for a typical SPA session is effectively "never," until the page is unloaded.

---

5. Edge Cases & Gotchas

5.1 Accidental Multiple Instances of a "Singleton"

The most common real-world bug: a developer intends a service to be a true app-wide singleton (`providedIn: 'root'`) but **also** lists it in a component's (or, historically, a lazy-loaded module's) `providers` array — often out of habit, copy-pasted boilerplate, or a misunderstanding that "providers must be declared explicitly." The moment that happens, that component subtree silently gets its own private instance, and state written by other parts of the app is invisible there. Symptoms: "the cart badge doesn't update," "my auth service seems to forget the logged-in user in this one screen." The fix is always the same: search for redundant `providers: [ServiceName]` entries and remove them if a genuine app-wide singleton is intended.

A more subtle historical variant: pre-Ivy (ViewEngine) lazy-loaded modules that listed a `root`-provided service **again** in their own `@NgModule({ providers: [...] })` would get a **second, module-scoped instance** for that lazy chunk, breaking the singleton assumption invisibly. Ivy's `providedIn: 'root'` mitigates a lot of this by not requiring module-level declarations at all, but explicit redundant declarations still cause it.

5.2 Memory Leaks from Services Holding Subscriptions

Because a `root`-scoped service lives for the entire application lifetime, **any subscription it creates and never cleans up lives forever too**. The most dangerous version of this: a service subscribes to something and, in the callback, holds a reference to a component instance (or an `Observable` originating from a component) — this pins that component in memory even after it's destroyed and removed from the DOM (a classic Angular memory leak: components that should be garbage collected are kept alive because a longer-lived singleton service holds a reference to them via an active subscription).

```
// BAD: leak-prone
@Injectable({ providedIn: 'root' })
export class NotificationService {
  constructor(private ws: WebSocketService) {
    // This subscription is created once and never torn down —
    // for a root service, that's usually fine for its OWN observable,
    // but is dangerous if it captures short-lived objects in the callback.
    this.ws.messages$.subscribe(msg => this.handle(msg));
  }
}
```

This specific example is usually harmless *if* `ws.messages$` itself never completes and the service is meant to live forever anyway — the real danger is when a **component-scoped** service (or the component itself) subscribes to a **long-lived** observable (a root service's stream, a global `fromEvent`, a `setInterval`-based observable, a `Router.events` stream) and never unsubscribes:

```
// BAD: component subscribes to a service's long-lived stream, never cleans up
@Component({ selector: 'app-widget', template: `...` })
export class WidgetComponent implements OnInit {
  constructor(private cart: CartService) {}

  ngOnInit(): void {
    this.cart.items$.subscribe(items => {
      // do something with items
    });
    // No corresponding unsubscribe -> this subscription outlives the component.
    // The Subject inside CartService retains a reference to this callback
    // (and therefore to `this`, the destroyed component), leaking memory
    // and potentially causing "still running" side effects after destroy.
  }
}
```

Correct pattern — always tie subscription lifetime to the component's own lifecycle:

```
@Component({ selector: 'app-widget', template: `...` })
export class WidgetComponent implements OnInit, OnDestroy {
```

```
  private destroy$ = new Subject<void>();

  constructor(private cart: CartService) {}

  ngOnInit(): void {
    this.cart.items$
      .pipe(takeUntil(this.destroy$))
      .subscribe(items => { /* ... */ });
  }

  ngOnDestroy(): void {
    this.destroy$.next();
    this.destroy$.complete();
  }
}
```

Modern equivalent using `takeUntilDestroyed()` (Angular 16+):

```
ngOnInit(): void {
  this.cart.items$
    .pipe(takeUntilDestroyed())
    .subscribe(items => { /* ... */ });
}
```

Or, simplest and safest for template-only consumption: use the `async` pipe, which subscribes/unsubscribes automatically in lockstep with the view's lifecycle — no manual cleanup code needed at all.

5.3 Other Common Gotchas

- **Constructor side effects at the wrong time**: because a `root` service's constructor runs at first injection (not at app bootstrap), if your service kicks off an HTTP call or a `setInterval` in its constructor, the timing of "when this first fires" is dictated by *whichever component happens to inject it first* — which can be a surprising, hard-to-reproduce ordering bug. Prefer an explicit `init()` method called deliberately, or an `APP_INITIALIZER` if it truly must run at bootstrap.
- **Mutating a `BehaviorSubject`'s emitted array/object in place**: `this.itemsSubject.value.push(x)` followed by `.next(this.itemsSubject.value)` will not reliably trigger change detection with `OnPush` components subscribed via `async`, because the reference didn't change. Always emit new array/object references (see §3.2's use of spread/`map`/`filter`).
- **Overusing component-level `providers`**: instantiating a "service" per component when the intent was actually just to share state defeats the entire purpose of DI-based communication — a common mistake among developers newer to Angular who add `providers: [SomeService]` reflexively to every component.
- **Confusing a service instance with a "static" utility class**: if a class has no state and no injected dependencies, ask whether it needs to be a DI-managed service at all, or whether a set of plain exported functions is simpler and avoids DI overhead/mocking ceremony entirely.
- **`providedIn: 'any'` surprises**: developers expect one instance across the whole app, but each lazy-loaded module that injects an `'any'`-scoped service gets its own instance — while eagerly-loaded code

all shares one instance. This split behavior is a frequent source of "it works in the eager route but not in the lazy one" bugs.

---

6. Interview Questions & Answers

Q1. What is a service in Angular, and why do we need `@Injectable()`?
A service is just a plain TypeScript class used to encapsulate logic (data access, shared state, utilities) outside of components. `@Injectable()` marks the class as a target for Angular's dependency injection: it lets Angular generate the metadata needed to resolve the class's own constructor dependencies, and (via `providedIn`) registers the class with an injector without needing a module's `providers` array. Technically a class with zero dependencies can work without the decorator, but it's fragile and considered bad practice to omit it.

Q2. What does `providedIn: 'root'` actually do, mechanically?
It attaches a static injectable definition (`ɵprov`) directly onto the class, telling Angular "register this class as a provider on the root `EnvironmentInjector`, with this factory function." Because the registration lives on the class rather than in a module's static list, unused services can be tree-shaken out of the final bundle. It also means the service behaves as an application-wide singleton, assuming no closer injector re-declares it.

> **Interviewer intent:** This checks whether the candidate understands tree-shakable providers versus the old NgModule-based `providers: []` registration, which is a common "what changed with Ivy" question.

Q3. Is an Angular service a "true" singleton like the classic Singleton design pattern?
No — it is a singleton **per injector**, not per process. Angular has a hierarchy of injectors (root `EnvironmentInjector`, per-lazy-module injectors, per-component `ElementInjector`s). A service is guaranteed to be one shared instance only within the scope of the injector that provided it. The same class can have multiple, independent instances simultaneously if it's provided at multiple points in the injector tree (e.g., once at root, and again in a component's `providers` array).

Q4. How would you design a service to share state between two sibling components that don't have a parent-child relationship?
Create a `providedIn: 'root'` service holding the shared state privately (e.g., in a `BehaviorSubject` or a `signal`), expose it as read-only (`.asObservable()` / `.asReadonly()`), and expose explicit mutation methods rather than allowing direct `.next()` calls from outside. Both sibling components inject the same service via their constructors; because it's root-scoped, DI hands them the identical instance, so one component's mutation is immediately visible to the other through the shared observable/signal — no `@Input` / `@Output` chain or common ancestor required.

> **Interviewer intent:** This is a design question probing whether the candidate can build the shared-state pattern from scratch, not just recite that "services can share state." Watch for candidates who expose the raw `Subject` publicly — that's a real design smell interviewers look for.

Q5. When would you deliberately scope a service to a component instead of root?
When you need a fresh, isolated instance per component instance rather than one shared instance app-wide — for example, a `<tab-group>` component repeated multiple times on a page, each needing its own "currently active tab" state. You add the service (without `providedIn`, or overriding a root-provided one) to that component's `providers` array; Angular then creates a separate instance per component instance, and it is automatically destroyed with the component.

Q6. What happens if a service is provided in `providedIn: 'root'` *and* also listed in a component's `providers` array?
The component-level `providers` entry creates a new provider record on that component's own `ElementInjector`. Angular's DI resolution walks the injector tree starting from the requesting component and stops at the first matching provider — so anything requesting the service from within that component's subtree gets the **component-scoped instance**, never reaching the root registration. This is a common source of "singleton isn't behaving like a singleton" bugs.

Q7. Are `root`-provided services instantiated at application bootstrap?
No. Instantiation is lazy and demand-driven: the injector only holds provider *definitions* (recipes) until the first actual injection request for that token occurs — whether from a component constructor, another service's constructor, or an `inject()` call. At that point the injector resolves the constructor dependencies (recursively, if needed), invokes the factory, and caches the resulting instance for all future requests from that injector or its descendants. A service that is never injected anywhere in a given session never has its constructor run.

> **Interviewer intent:** Tests whether the candidate conflates "registered" with "instantiated" — a distinction that explains subtle bugs like constructor side effects firing at unexpected, hard-to-predict times.

Q8. How do you unit test a service that depends on `HttpClient`?
Use `TestBed` to configure a testing module that provides `provideHttpClientTesting()` (or `HttpClientTestingModule` in older Angular), then `TestBed.inject(YourService)` to get an instance resolved through the same DI mechanism as production, and `TestBed.inject(HttpTestingController)` to assert on and flush mock HTTP requests instead of hitting a real backend:

```
TestBed.configureTestingModule({
  providers: [MyService, provideHttpClient(), provideHttpClientTesting()]
});
const service = TestBed.inject(MyService);
const httpMock = TestBed.inject(HttpTestingController);
service.getAll().subscribe(data => expect(data.length).toBe(2));
const req = httpMock.expectOne('/api/products');
req.flush([{ id: 1 }, { id: 2 }]);
httpMock.verify();
```

Q9. Do you always need `TestBed` to test a service?
No — if a service has no injected dependencies (or only dependencies you're comfortable instantiating/mocking by hand), you can simply `new` it up directly in a test, which is faster and avoids Angular's test machinery entirely:

```
const service = new CartService();
service.addItem({ productId: 1, name: 'Widget', quantity: 1, price: 9.99 });
expect(service.itemCount).toBe(1);
```

`TestBed` becomes valuable once the service needs Angular-managed dependencies (`HttpClient`, `Router`, other injectable services) resolved and mocked the way production DI would.

Q10. What's a subtle memory leak involving services, and how do you prevent it?
A `root`-scoped service lives for the entire application session, so any subscription it holds lives just as long. The dangerous case is the reverse direction: a short-lived component subscribes to a long-lived observable exposed by a root service (or a global stream) and never unsubscribes on destroy — the `Subject`/observable inside the service retains a reference to the subscriber callback, which closes over `this` (the component), preventing it from being garbage collected even after it's removed from the DOM. Prevent it by tying subscriptions to the component's lifecycle: `takeUntil(this.destroy$)` combined with unsubscribing/completing in `ngOnDestroy`, the newer `takeUntilDestroyed()` operator, or — simplest — using the `async` pipe in the template so Angular manages subscribe/unsubscribe automatically.

> **Interviewer intent:** This is one of the most practically important Angular interview questions — many candidates can define a service but can't explain how careless subscription management inside/around services causes real production memory leaks. Strong answers name a specific cleanup mechanism, not just "you should unsubscribe."

Q11. Why should you expose a `BehaviorSubject` as `.asObservable()` rather than the `Subject` itself?
If you expose the raw `Subject`/`BehaviorSubject`, any consumer can call `.next()` on it directly, bypassing the service's own validation/business logic and creating an unpredictable, hard-to-trace set of places that mutate shared state. Exposing `.asObservable()` gives consumers a read-only stream — they can subscribe but cannot push values — forcing all mutations through explicit, named methods on the service (`addItem()`, `clear()`, etc.), which keeps state changes auditable and centralizes validation.

Q12. What is `providedIn: 'any'` and how does its instantiation behavior differ from `'root'`?
`providedIn: 'any'` gives every lazy-loaded module (i.e., every separate `EnvironmentInjector` created for a lazy route) its **own** instance of the service, while all eagerly-loaded parts of the application (which share the root injector) still get a single common instance. This differs from `'root'`, which always yields exactly one instance for the whole app regardless of lazy loading. It's a niche option, mainly useful when you deliberately want per-lazy-chunk isolation of some piece of state, and a frequent source of confusion when developers assume it behaves identically to `'root'`.

Q13. How does circular dependency between two services manifest, and how do you fix it?
If `ServiceA`'s constructor injects `ServiceB`, and `ServiceB`'s constructor injects `ServiceA`, Angular's DI resolution — which is a depth-first walk that must fully construct one dependency before it can construct the one that needs it — cannot complete either constructor first, and Angular throws `NG0200: Circular dependency in DI detected`. The fix is architectural: extract the shared logic both services need into a third service that both `A` and `B` depend on (breaking the cycle), or, where truly unavoidable, use Angular's `Injector` to lazily resolve one of the two dependencies at call-time rather than at construction-time (an anti-pattern to reach for only as a last resort, since it usually signals the two services are too tightly coupled).

Q14. Why prefer several small, focused services over one large service that handles many concerns?
A large service handling unrelated concerns (say, authentication, logging, and cart management all in one class) becomes hard to unit test in isolation (every test needs to mock everything the "god service" touches, even features unrelated to what's under test), hard to reason about (any change risks unrelated regressions), and creates unnecessary coupling for any component that only needs one slice of its functionality but ends up depending on the whole thing. Composing small, single-responsibility services (e.g., `AuthService`, `CartApiService` that injects `AuthService`) keeps each piece independently testable and mockable, and mirrors the same single-responsibility principle applied to components.

---

7. Quick Revision Cheat Sheet

- **Service** = plain class doing logic/state work outside components; `@Injectable()` = marks it as a DI target and (with `providedIn`) tree-shakably registers it with an injector.
- `providedIn: 'root'` → one instance for the whole app (singleton *per root injector*, not a language-level singleton).
- `providedIn: 'any'` → one instance per lazy-loaded module's injector; eager code shares one root instance.
- **Component** `providers: [X]` → new instance of `X` per component instance; **overrides/shadows** any root registration for that subtree; destroyed with the component.
- DI resolution walks the injector tree **upward** from the requester; first matching provider wins.
- Instantiation is **lazy**: constructor runs on first injection request, not at bootstrap; instance is then cached on that injector.
- **Shared-state pattern**: private `BehaviorSubject` / `signal` + public `.asObservable()` / `.asReadonly()` + explicit mutation methods — never expose the raw `Subject`.
- **Composition**: services injecting other services is normal; watch for circular dependencies (`NG0200`) and "god services."
- **Root-scope** for app-wide singletons/stateless utilities; **component-scope** for per-UI-instance isolated state with automatic cleanup.
- **Memory leaks**: long-lived (root) service streams + short-lived component subscribers that never unsubscribe = leaked components. Fix with `takeUntil` + `ngOnDestroy`, `takeUntilDestroyed()`, or the `async` pipe.
- **Testing**: `new ServiceClass()` for dependency-free services; `TestBed.inject()` + `HttpTestingController` / `provideHttpClientTesting()` when the service has Angular-managed dependencies.
- Always emit **new references** (spread/map/filter) from state services — mutating in place breaks `OnPush` change detection downstream.

Created By - Durgesh Singh